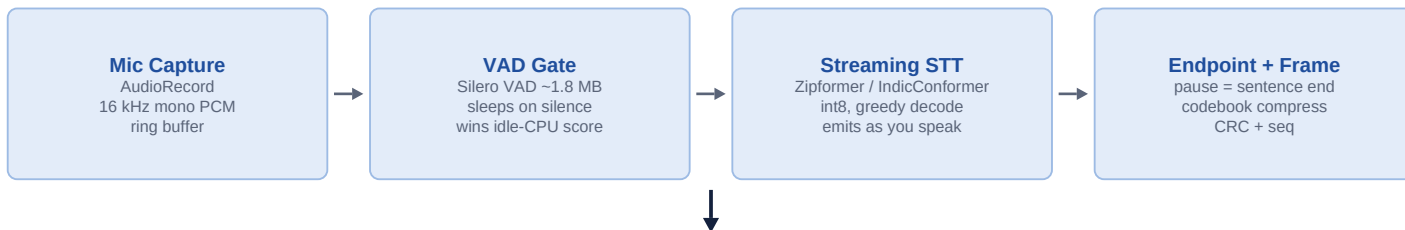


CORE IDEA Voice is too heavy for satellite / LoRa / HF links. So we send **TEXT** on the wire and rebuild **VOICE** at both ends.

Speech in → STT → ~100 byte text frame → radio → TTS → speech out. Everything runs on-device, fully offline, in 10 Indian languages.

1. Sender — Speech to Text

runs on phone CPU, no network

**2. Transport — one interface, three backends**

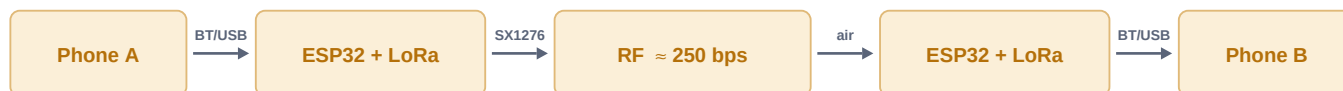
the phone never speaks LoRa directly

A DIRECT phone to phone

range 10–100 m • demo + the verification loop the PS asks for

**B RADIO BRIDGE** long haul

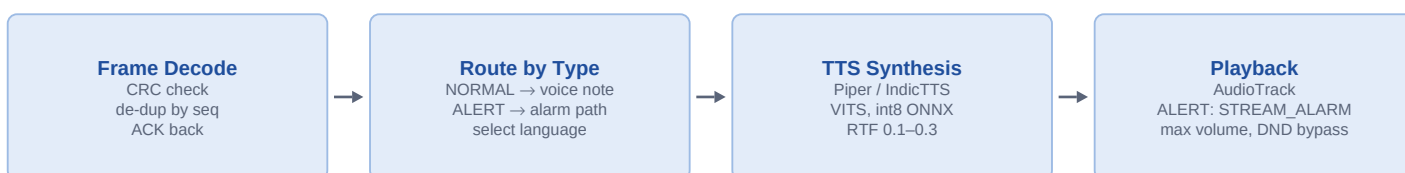
range 2–5 km • 866 MHz India ISM band • SF12 ≈ 250 bps

**C GATEWAY** shared radio

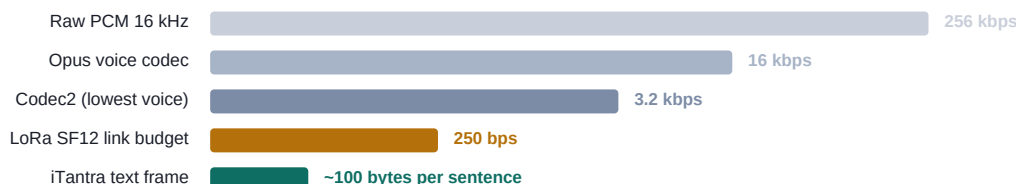
phones with no module borrow one that has it • ALERT frames get queue priority

**WIRE FRAME** — identical over Bluetooth, Wi-Fi and LoRa`[ver:4b][type:4b][src:8b][dst:8b][lang:8b][seq:8b][len:8b][payload][crc16]` `dst=0xFF broadcast` `type = NORMAL | ALERT | ACK`**3. Receiver — Text to Speech**

identical app, opposite mode

**4. Why text on the wire**

cost of one spoken sentence, log scale

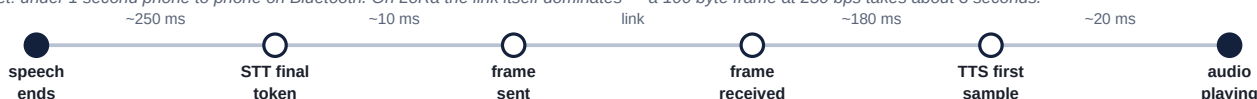


Voice cannot fit through a 250 bps link. Text can, with room to spare. STT and TTS make a text-only channel feel like voice.

5. End-to-end latency budget

measured live in-app and shown on the metrics HUD

Target: under 1 second phone to phone on Bluetooth. On LoRa the link itself dominates — a 100 byte frame at 250 bps takes about 3 seconds.



CONSTRAINTS

Open-source only • no proprietary voice SDKs • fully offline • TFLite / PyTorch Mobile class frameworks • must run on low & mid-range phones

Rules out Google / Azure / AWS Speech, ElevenLabs, and Android's built-in SpeechRecognizer (closed source, Play Services dependent, often needs network).

APPLICATION

Android App

Language	Kotlin
UI	Jetpack Compose
Min SDK	26 (Android 8.0)
Concurrency	Coroutines + Flow
Background	Foreground Service
Audio in / out	AudioRecord / AudioTrack
ABI	arm64-v8a only

INFERENCE

On-Device ML Runtime

Primary	sherpa-onnx (Android AAR)
Engine	ONNX Runtime Mobile
CPU kernels	XNNPACK
Accelerator	NNAPI delegate
Alternative	TFLite / PyTorch Mobile
Quantization	int8, ~4x smaller
Why	STT + TTS + VAD in one dep

MODELS

Pretrained, Not Trained

VAD	Silero VAD ~1.8 MB
STT primary	Streaming Zipformer ~70 M
STT Indic	AI4Bharat IndicConformer
STT fallback	Vosk small models
TTS primary	Piper (VITS) 20–60 MB
TTS Indic	AI4Bharat IndicTTS / IndicF5
Baseline only	Whisper-tiny int8 ~40 MB
Source	Hugging Face, permissive

TRANSPORT

Getting Bytes Out

Backend 1	Bluetooth RFCOMM (SPP)
Backend 2	Wi-Fi Direct + UDP multicast
Backend 3	USB serial (OTG)
USB library	usb-serial-for-android
BT piconet cap	7 active slaves
Avoided	Nearby Connections, Wi-Fi Aware
Addressing	src / dst in frame header
Contention	listen-before-transmit + backoff

OFFLINE TOOLING

Laptop / Python

Quantize	onnxruntime.quantization
WER metric	jiwer
Test sets	Kathbath, IndicSUPERB, CV
Augmentation	librosa + numpy (noise)
RAM profiling	adb dumpsys meminfo
CPU profiling	adb top / BatteryStats
Output	per-language WER / RTF / size

OPTIONAL DEMO

Radio Bridge Prop

Board	ESP32 (WROOM-32 = BT Classic)
Caution	S3 / C3 are BLE-only → use USB
Radio	SX1276 / SX1262 LoRa
Firmware	Arduino + RadioLib
Link	BluetoothSerial or USB serial
Band	865–867 MHz (India, WPC/DoT)
Config	SF12 / 125 kHz ≈ 250 bps

FOOTPRINT

Size & Memory Plan

Bundled in APK	1 language pair only
Rest of the models	downloaded on demand
Resident at runtime	1 STT or 1 TTS, never both
STT after int8	~40–70 MB
TTS voice	~20–60 MB
VAD	~1.8 MB, always on
Idle CPU	VAD only, model asleep
Target APK	under 100 MB

DELIBERATELY NOT USED

And Why

React Native / Flutter	bridge cost on a real-time DSP path
Google / Azure / AWS STT	cloud, closed — breaks two rules
Android SpeechRecognizer	Play Services, often needs network
ElevenLabs & commercial TTS	proprietary SDK, explicitly barred
Nearby Connections	Play Services; raw sockets stay clean
Wi-Fi Aware (NAN)	patchy on low-end phones
Beam search decoding	cost multiplier, tiny WER gain
Training our own models	months of GPU time we do not have

WHAT THE STACK IS OPTIMISED FOR

40% Accuracy

best Indic model per language

20% Latency

streaming STT, greedy decode

20% Efficiency

int8 + VAD gating + lazy load

20% Architecture

pluggable transport, live metrics

FIRST THREE STEPS

1. Build the sherpa-onnx Android streaming-ASR sample with one English model. Prove the toolchain before writing any own code.
2. Benchmark every candidate STT and TTS model per language on a real low-end phone. Record WER, RTF and size in one sheet.
3. Quantize the winners to int8, re-benchmark, then lock the model choices. Only after that touch transport, UI or hardware.